



PROJECT DOCUMENTATION

Flashpod

Reaction-Time Game System

PLATFORM	nRF52840 Dongle · Zephyr RTOS · nRF Connect SDK
DOCUMENT VERSION	1.0
DATE	July 2026
PODS SUPPORTED	1 hub + up to 3 remote pods

Contents

01	Overview	What Flashpod is and how a round of play works
02	System Architecture	Hub-and-spoke design and responsibilities
03	Hardware & Pin Mapping	Component list and hub pinout diagram
04	Software Modules	Firmware structure at a glance
05	Communication Model	How the hub, app, and pods stay in sync
06	Game Modes	Single-Pod Reaction and Odd One Out, compared
07	Configuration & Build	Firmware options and getting a build onto the hub

01 Overview

Flashpod is a reaction-time training system built around a small network of light-up pods — tap the pod that lights up, and the system tells you how fast you were.

One pod in the set is the **hub**: it is the only device that talks directly to a companion phone app, and it coordinates every other pod over its own wireless links. The hub also carries its own light ring, tap sensor, and buzzer, so it can play as Pod 1 while also running the game.

A typical session looks like this: the player opens the app, picks a mode and a colour, and starts a round. The hub picks a pod (or several, depending on the mode), lights it up, and starts a timer. As soon as a tap is detected — on the hub itself or on a remote pod — the result is measured, a tone plays, and the outcome is sent back to the app. The next round then begins automatically.

Up to 4 pods total

2 game modes

Millisecond-accurate timing

Wireless, battery-friendly pods

WHY A HUB-AND-SPOKE DESIGN

Keeping all the game logic on one pod means the remote pods can stay simple and cheap — they only need to light up and report a tap — while the player always gets one consistent, authoritative score.

02 System Architecture

Flashpod uses a **hub-and-spoke topology**. The hub is a dual-role Bluetooth Low Energy device: it acts as a **peripheral** to the phone app (the phone initiates that connection) and as a **central** to every remote pod (the hub initiates those connections). This gives the system one clear "brain" and keeps every other component lightweight.

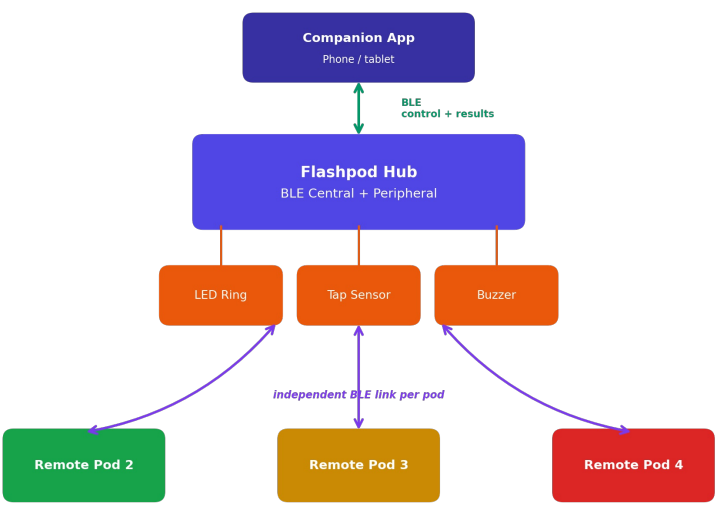


Figure 1 — Flashpod's hub-and-spoke architecture. The hub is the only node that talks to the phone; every pod link is independent.

Responsibilities

- **Companion app** — lets the player choose a mode and colour, start or stop play, and view results and reaction times as they come in.
- **Flashpod Hub** — owns all game logic: which pod(s) light up, in what colour, for how long, and whether a tap counts as correct. It is the single source of truth for scoring.
- **Remote pods** — autonomous responders. Each lights up on command and reports back either a tap time or a timeout; they don't need to know anything about game rules.

DESIGN NOTE

Because scoring is centralised on the hub rather than split across pods, results shown on the phone always match what actually happened physically — there's no risk of two pods disagreeing about who won a round.

03 Hardware & Pin Mapping

The hub is built on an nRF52840 dongle. It carries three pieces of onboard hardware in addition to its radio, plus a physical mode button for switching game modes without touching the phone.

Component List

Component	Description	Interface
nRF52840 Dongle	Main controller — runs the game, bridges the phone and the remote pods	USB / BLE
LED Ring (16× WS2812B)	Individually addressable RGB LEDs used to prompt a tap	SPI (data line)
Tap Sensor	Contactless proximity sensor for touch-free tap detection	GPIO, interrupt-driven
Buzzer	Audio feedback for correct taps, wrong taps, and timeouts	GPIO (direct drive)
Mode Button	Physical switch between Mode 1 and Mode 2	GPIO, interrupt-driven
Remote Pods (×3)	Independent battery pods, each with its own light and tap sensor	BLE

Hub Pinout

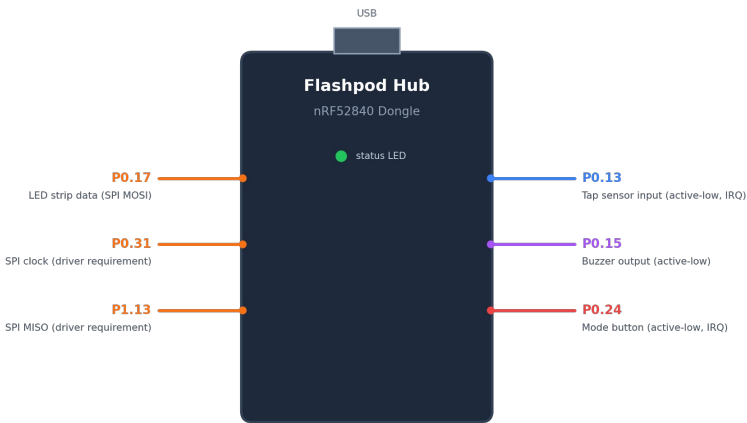


Figure 2 — GPIO pin assignments on the Flashpod Hub.

Pin	Function	Notes
P0.17	LED ring data line	SPI MOSI, drives the WS2812B strip
P0.31	SPI clock	Required by the SPI driver; unused by the LEDs themselves
P1.13	SPI MISO	Required by the SPI driver; unused
P0.13	Tap sensor input	Active-low, internal pull-up, interrupt-driven
P0.15	Buzzer output	Active-low (driven low to sound)
P0.24	Mode button input	Active-low, internal pull-up, interrupt-driven

04 Software Modules

The hub firmware is split into small, focused modules rather than one monolithic file. This keeps each piece of hardware easy to reason about and test on its own.

Module	Responsibility
LED Driver	Initialises the LED ring, sets all pixels to a colour and brightness, and turns them off between rounds.
Tap Sensor Driver	Reads the tap sensor in either polling or interrupt mode, and can be enabled or disabled so it doesn't wake the system when no round is active.
Buzzer Driver	Produces short and long tones for correct taps, wrong taps, and timeouts. Beep requests are queued so a playing tone never blocks game logic.
Main Application	Owns Bluetooth connectivity (both roles), the game loop, and the command interface shared by the app and a local debug console.

Concurrency at a Glance

Several lightweight background tasks run concurrently on the hub — one watches the local tap sensor, one runs the game loop, one plays queued buzzer tones, and one listens for debug commands. Splitting the work this way means a blocking operation, like a 100 ms buzzer tone, never delays tap detection or the next round of play.

ROBUSTNESS

A safety margin is added to every round's timeout wait, so a remote pod's result arriving a little late over BLE never gets mistaken for a lost message.

05 Communication Model

All communication — phone-to-hub and hub-to-pod — runs over a standard Bluetooth serial-style profile, exchanging short, plain-text status messages. This keeps every link easy to inspect and debug, including from a desktop terminal.

What gets exchanged

- **App → Hub:** start or stop a round, choose a colour or game mode, and set the round timeout.
- **Hub → App:** which pod lit up, reaction times in milliseconds, misses, timeouts, and mode or timeout changes.
- **Hub → Pod:** a light-up command with a target colour, or an all-off command.
- **Pod → Hub:** a reaction time on a successful tap, or a timeout notice.

Because every message is a short text line, the same debug console used for development can issue the same commands as the phone app, which makes bench-testing a pod change straightforward without needing the full app.

06 Game Modes

Flashpod ships with two modes that exercise different skills. Either can be selected from the app, or cycled with the physical mode button on the hub.

MODE 1

Single-Pod Reaction

One pod lights up at random in a chosen colour. The player taps it as fast as possible, and the hub reports the reaction time in milliseconds. Rounds repeat automatically.

- 1 active pod per round
- 6 selectable colours
- Configurable timeout (1–60s)
- Tests: raw reaction speed

MODE 2

Odd One Out

Every connected pod lights up together, but one shows a different colour. The player must spot and tap that pod — tapping any other pod counts as a miss.

- All connected pods active
- 3-colour pool, randomised
- Wrong-tap feedback (miss tone)
- Tests: visual discrimination + speed

Mode Comparison

Feature	Mode 1 — Single-Pod	Mode 2 — Odd One Out
Active pods per round	1 (random)	All available (2–4)
Primary skill	Reaction speed	Discrimination + speed
Wrong tap possible	No	Yes
Minimum pods required	1	2
Auto-loop between rounds	Yes	Yes

07 Configuration & Build

The firmware is configured through the standard Zephyr Kconfig system. At a high level, the build enables:

- **Bluetooth** in dual peripheral + central role, with enough simultaneous connections for the phone and three remote pods.
- **LED strip & SPI support**, for driving the WS2812B ring.
- **GPIO**, for the tap sensor, buzzer, and mode button.
- **Hardware RNG**, used for random pod and colour selection.
- **Serial console**, for local debugging alongside the phone app.

Building & Flashing

The project builds with the standard nRF Connect SDK / Zephyr toolchain (`west`, `CMake`, and `Ninja`) targeting the nRF52840 dongle board definition. The dongle's USB bootloader accepts a generated DFU package, which can be flashed over USB without any extra debug hardware.

PREREQUISITES

nRF Connect SDK (v2.x or compatible), the Zephyr toolchain, and either `nrfutil` or nRF Connect Programmer for flashing.